Project - High Level Design

On
Dockerized Sports Rust Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Chetan Oswal	EN22CS301295
2	Irya Patni	EN22CS301436
3	Uday Dubey	EN22EL301058
4	Himani Jaiswal	EN22CS301423

Group Name: Placed BtechGroup07

Project Number: DO-50

Industry Mentor Name:

University Mentor Name :

Academic Year: 2025-2026

Table of Contents

1. Introduction 3
2. Scope of the Document 3
3. Intended Audience 4
4. System Overview 4
5. System Design 5
6. Application Design 6
7. Process Flow 6
8. Information Flow 7
9. Components Design 8
10. Key Design Considerations 9
11. API Catalogue 10
12. Data Design 10
13. Data Model 10
14. Data Access Mechanism 11
15. Data Retention Policies 11
16. Data Migration 11
17. Interfaces 12
18. State and Session Management 12
19. Caching 12
20. Non-Functional Requirements 12
21. Security Aspects 12
22. Performance Aspects 13
23. References 13

1. Introduction

Building scalable and secure web applications requires proper separation of concerns, efficient backend processing, and reliable data management. Many applications fail to maintain performance and security when deployed without a structured architecture.

To address this, we developed the Dockerized Sports Rust Service, a fully containerized platform based on a 3-tier microservice architecture. The system separates the frontend, backend, and database into independent services, ensuring better scalability, maintainability, and security. It uses modern technologies like Rust for high performance and Docker for containerization.

The main aim of this project is to demonstrate how to design, containerize, and deploy a robust distributed system using modern DevOps practices.

2. Scope of the Document

This document explains the overall design of our system. It includes:

- How the system works
- The architecture and components
- Data handling methods
- APIs used
- Security and performance considerations

This document provides a high-level understanding of the system structure and design.

3. Intended Audience

This document is prepared for:

- Faculty members and evaluators
- Industry mentor
- Developers working on the project
- Anyone who wants to understand the system design

4. System Overview

The Dockerized Sports Rust Service is a containerized web platform designed using a 3-tier architecture.

1. The user accesses the application through a browser interface.
2. The request is first handled by an Nginx frontend server acting as a reverse proxy.
3. The request is forwarded to the Rust backend API for processing.
4. The backend interacts with the PostgreSQL database to fetch or store data.
5. The processed response is sent back through the backend to the frontend and displayed to the user.

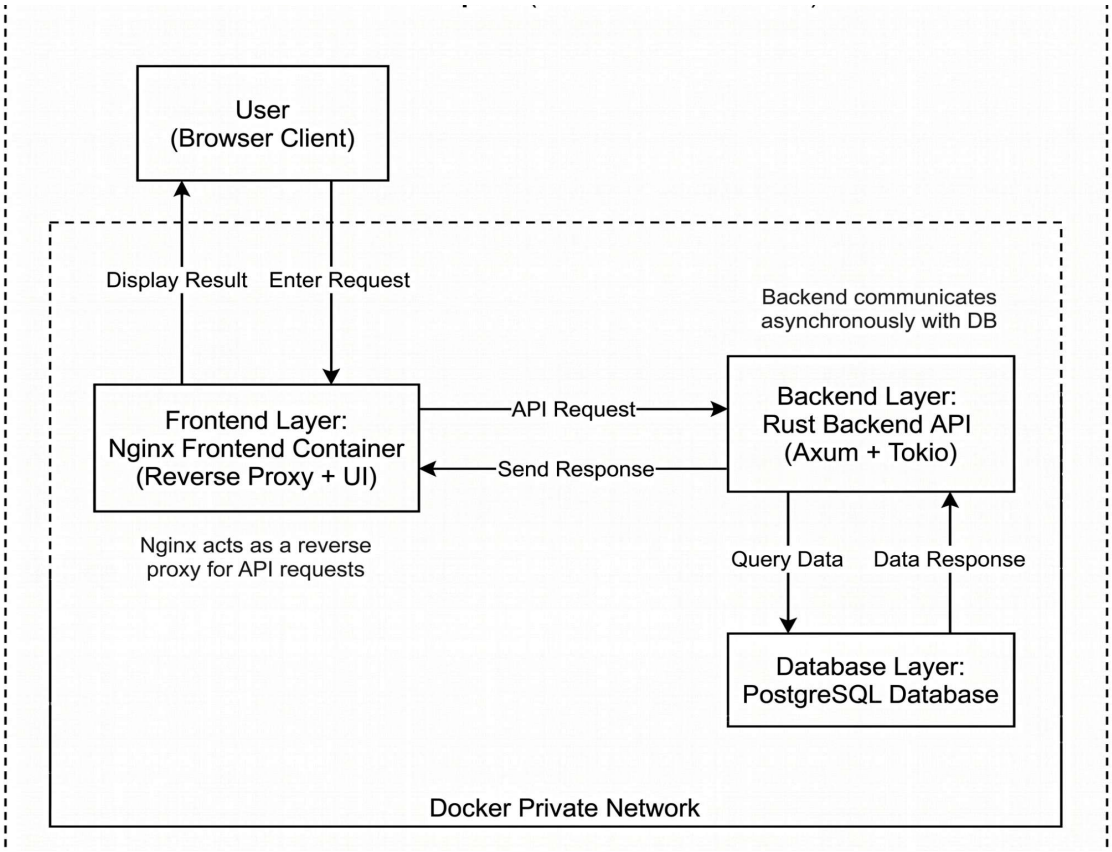
This structured flow ensures efficient communication, improved security, and scalability of the system.

5. System Design

Our system follows a modular 3-tier architecture. It consists of:

- User Interface (Frontend)
- Backend API Service
- Database Layer
- Reverse Proxy Layer
- Container Orchestration System

Each component performs a specific task, and together they ensure smooth system operation. The system uses Docker Compose for orchestration with service dependencies and health checks to ensure proper startup order.



6. Application Design

The application is designed as a containerized web-based system.

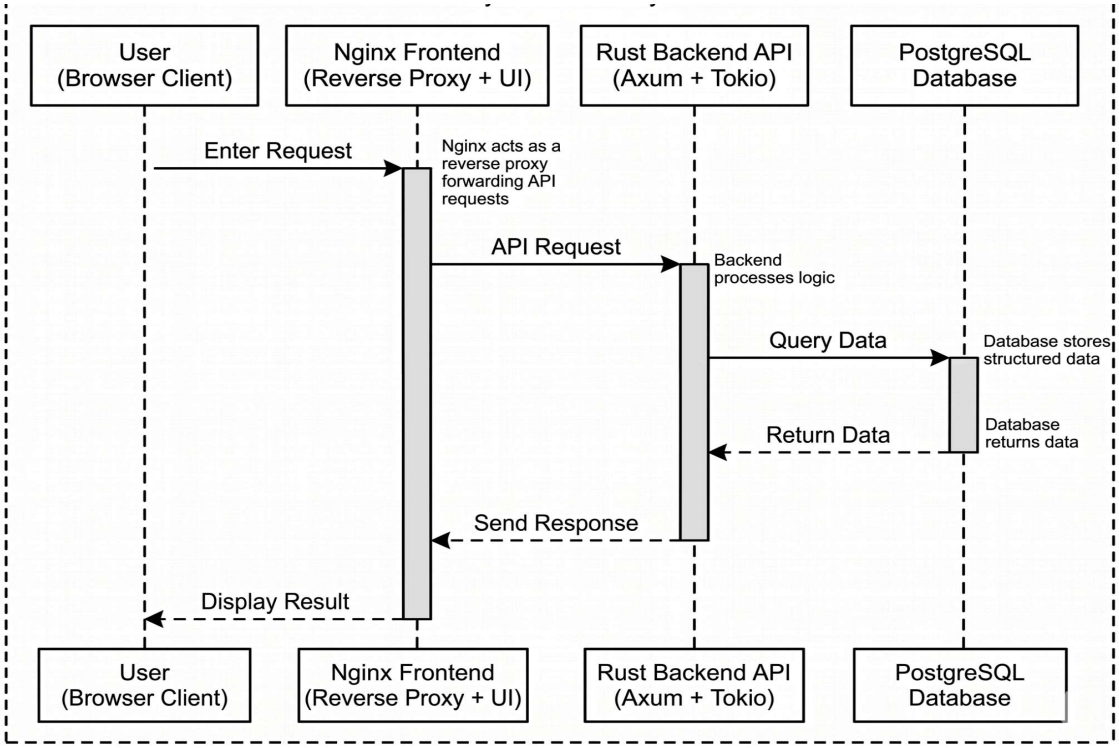
- A frontend interface served using Nginx
- A backend service built using Rust (Axum + Tokio)
- A PostgreSQL database for data storage
- Docker containers to isolate each service
- Docker Compose for orchestration and service management

The system uses reverse proxy routing and container-based deployment to ensure efficient communication and scalability.

7. Process Flow

The process works in the following steps:

1. The user sends a request through the browser.
2. The request is received by the Nginx frontend server.
3. Nginx forwards API requests to the Rust backend service.
4. The backend processes the request and interacts with the PostgreSQL database.
5. The database returns the required data to the backend.
6. The backend generates the response and sends it back to the frontend.
7. The final result is displayed to the user.

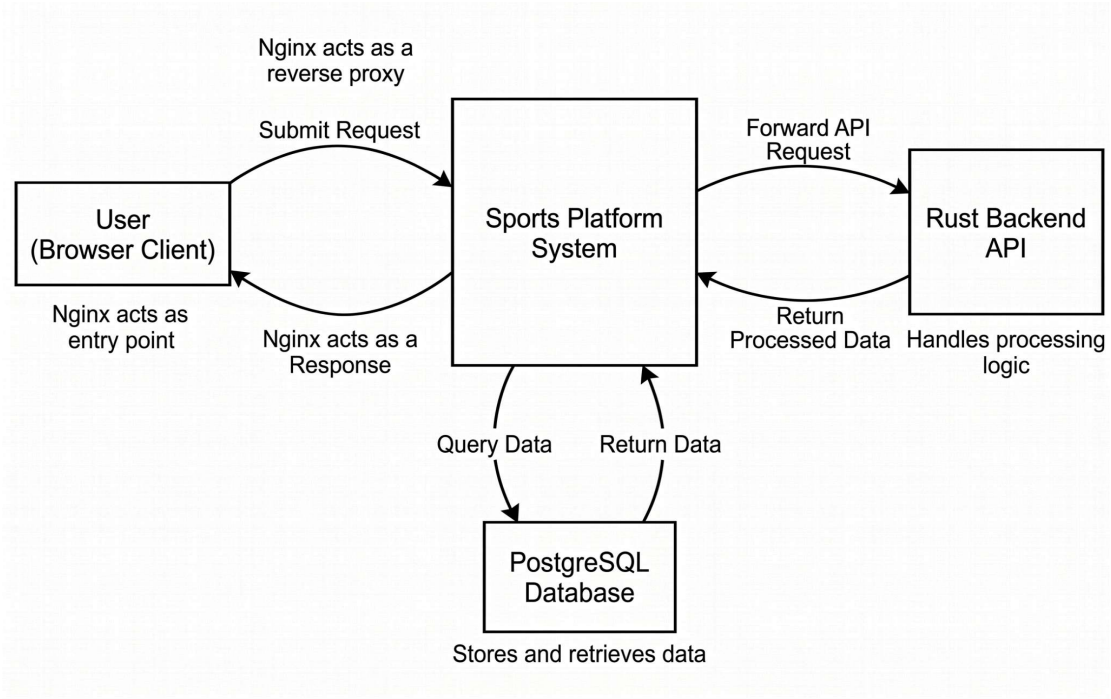


8. Information Flow

The information flows as follows:

User → Nginx Frontend → Rust Backend API
Rust Backend → PostgreSQL Database → Data Response
Backend → Nginx → User Response

The backend processes incoming requests, retrieves or updates data from the database, and sends the final response back through the frontend.



9. Components Design

1. User Interface

Allows users to interact with the application and view results.

2. Backend Module

Handles request processing and business logic.

3. Reverse Proxy (Nginx)

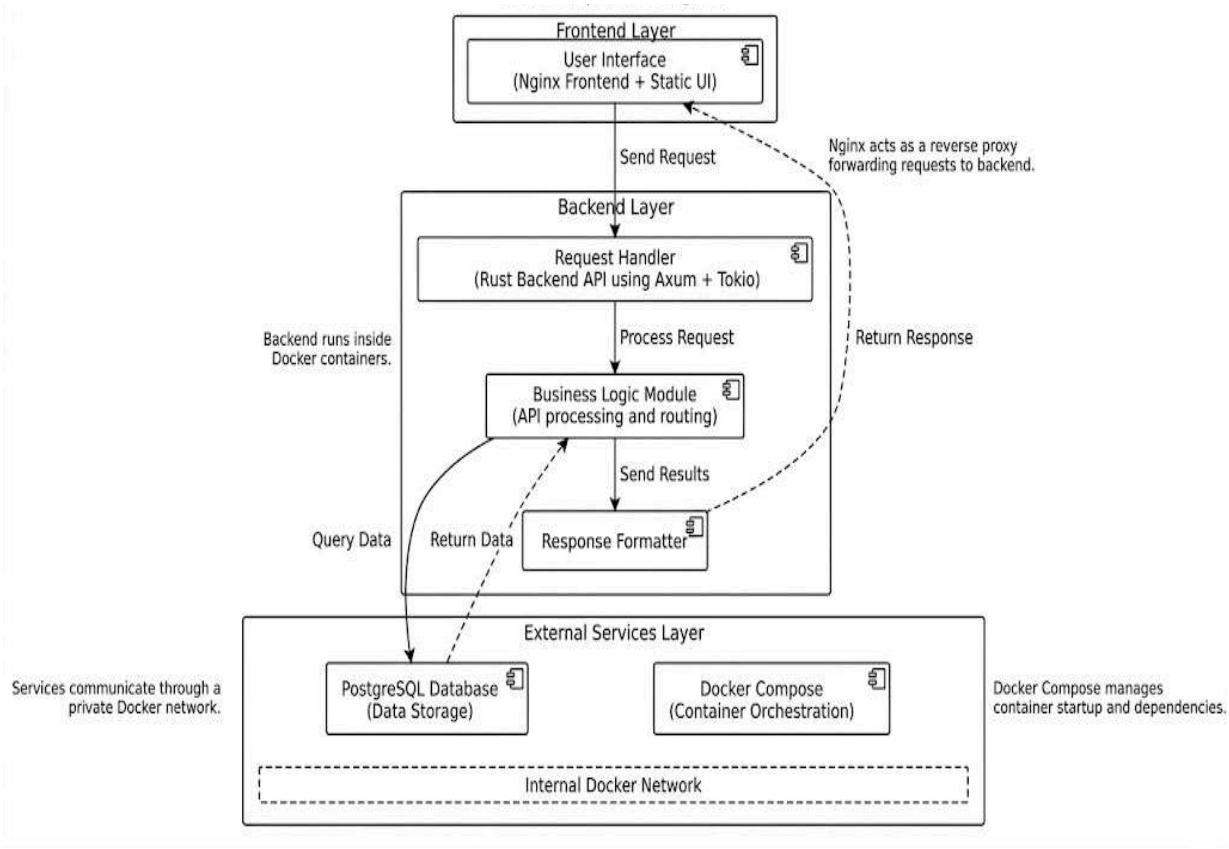
Routes incoming requests and forwards API calls securely.

4. Database Module (PostgreSQL)

Stores and manages application data.

5. Container Management (Docker)

Ensures isolated and scalable deployment of all services.



10. Key Design Considerations

While designing the system, we focused on:

- Keeping the architecture modular and scalable
- Ensuring high performance using Rust
- Maintaining secure communication using reverse proxy
- Designing for containerized deployment using Docker
- Supporting future scalability and cloud deployment

11. API Catalogue

Rust Backend API

- Handles all application logic and data processing
- Receives HTTP requests from the frontend
- Returns JSON responses

PostgreSQL Database

- Stores application data
- Handles queries from the backend
- Returns structured data

12. Data Design

The system works with structured data.

Types of data used:

- User requests
- API responses
- Sports-related data (teams, records)
- Database entries

The system maintains data in a structured relational format.

13. Data Model

Basic structure:

Request

- Request ID
- Request Data

Sports Data

- Team ID
- Team Name
- Details

Response

- Response ID
- Output Data
- Timestamp

14. Data Access Mechanism

- Communication between components is handled through APIs
- Data is transferred using JSON format
- Secure connections are maintained between services within the Docker network

15. Data Retention Policies

- User requests are not permanently stored
- Database stores only required application data
- No sensitive personal information is stored

16. Data Migration

- Currently, data migration is minimal due to local container storage
- In future, cloud database migration can be implemented for scalability

17. Interfaces

External Interfaces

- Web Browser
- HTTP Requests

Internal Interfaces

- Frontend to Backend communication
- Backend to Database communication

18. State and Session Management

- Each request is handled independently
- The system follows a stateless backend architecture
- Session handling is minimal and request-based

19. Caching

- Caching can be implemented at the backend or proxy level
- This reduces repeated database queries and improves performance

20. Non-Functional Requirements

The system should:

- Respond quickly
- Be reliable and secure
- Be scalable
- Handle multiple users efficiently
- Maintain high availability

21. Security Aspects

- Secure communication through HTTP/HTTPS
- Isolation using Docker containers
- Restricted internal network access
- Reverse proxy protection using Nginx

22. Performance Aspects

- High performance using Rust backend
- Efficient database queries
- Optimized container deployment
- Reduced latency through internal networking

23. References

- Rust Documentation
- Docker Documentation
- PostgreSQL Documentation
- Nginx Documentation
- DevOps and Microservices Architecture Resources